Journal of Discrete Algorithms 52–53 (2018) 156–167

Contents lists available at ScienceDirect

Journal of Discrete Algorithms

www.elsevier.com/locate/jda

Linear ordering based MIP formulations for the vertex separation or pathwidth problem ☆

Sven Mallach

Dept. of Mathematics and Computer Science, Universität zu Köln, Cologne, Germany

ARTICLE INFO

Article history:
Available online 19 November 2018

Keywords:
Pathwidth
Vertex separation
Mixed integer programming

ABSTRACT

We consider the task to compute the pathwidth of a graph which is known to be equivalent to solving the vertex separation problem. The latter is naturally modeled as a linear ordering problem with respect to the vertices of the graph. Present mixed-integer programs for the vertex separation problem express linear orders using either position or set assignment variables. However, as we show, the lower bound on the pathwidth obtained when solving their linear programming relaxations is zero for any directed graph. This is one reason for their limited utility in solving larger instances to optimality. We then present a new formulation that is based on conventional linear ordering variables and a slightly different perspective on the problem. Its relaxation provably delivers non-zero lower bounds for any graph whose pathwidth is non-zero. Further structural results for and extensions to this formulation are discussed. Finally, an experimental evaluation of three mixed-integer programs, each representing one of the different yet existing modeling approaches, displays their potentials and limitations when used to solve the problem to optimality in practice.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

A tree decomposition of an undirected graph $G = (V, E)$ is a collection of sets $X_i \subseteq V$, $i \in I$, along with a tree $T = (I, F)$ such that (a) $\bigcup_{i \in I} X_i = V$, (b) there is a set X_i, $i \in I$, with $\{v, w\} \subseteq X_i$ for each $\{v, w\} \in E$, and (c) for each $v \in V$, the tree-edges F connect all tree-vertices $i \in I$ where $v \in X_i$ [31]. The width of a tree decomposition is defined as $\max_{i \in I} |X_i| - 1$ and the treewidth $tw(G)$ of G is the minimum width among all its tree decompositions. Path decompositions and the pathwidth $pw(G)$ of an undirected graph G are defined analogously, just now requiring that T is actually a path [5]. It follows directly that $tw(G) \leq pw(G)$.

Both width parameters are of theoretical as well as of practical interest. While deciding whether an arbitrary graph has pathwidth at most k is itself $\mathcal{NP}$-complete [2,27], many $\mathcal{NP}$-complete problems on graphs can be solved efficiently on instances of known constant bounded tree- respectively pathwidth. For example, Arnborg and Proskurowski gave linear time algorithms for the vertex cover, independent set, k-colorability, Hamiltonian circuit and further combinatorial problems in this case [1]. Typically, algorithms based on tree or path decompositions are fixed-parameter algorithms [8] or follow the dynamic programming [4] paradigm, i.e., their running times have constant factors that are exponential in the tree- or pathwidth. It is also known that for a graph with $|V| = n$, its pathwidth is at most $\mathcal{O}(\log n)$ times its treewidth [6] which can be used for the construction of approximation algorithms. In this paper, we deal with computing the exact pathwidth

☆ A conference proceedings version of this article has appeared in 'Combinatorial Algorithms', LNCS, Vol. 10765, Springer, 2017, pp. 327–340.

E-mail address: mallach@informatik.uni-koeln.de.

https://doi.org/10.1016/j.jda.2018.11.012
1570-8667/© 2018 Elsevier B.V. All rights reserved.

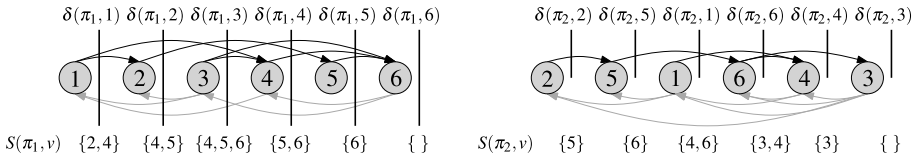


Fig. 1. A digraph drawn according to two different linear orders, $\pi_1 = \langle 1, 2, 3, 4, 5, 6\rangle$ and $\pi_2 = \langle 2, 5, 1, 6, 4, 3\rangle$, with illustrations of the associated cuts and separations.

of a graph and exploit the fact that the (directed) pathwidth of a (directed) graph is equivalent to its (directed) vertex separation number [24,34]. Besides the usefulness for practically exploring the pathwidth of certain graph classes (and thus potentially supporting or falsifying conjectures on their pathwidth), direct applications of the tree- and the pathwidth problems exist, among many others, in the context of VLSI design [15] and register allocation [10]. The directed vertex separation problem has, e.g., an application in optical communication networks [32]. For an overview of the several other (equivalence) relations with other graph-theoretical problems as well as applications, we refer to [5,6,16].

Exact approaches to compute the pathwidth have been studied rather rarely, especially compared to the treewidth. They can be roughly divided into enumerative [25,26,33], fixed-parameter [7,12,19,22] and combinatorial branch-and-bound algorithms [14,20], and finally mixed-integer programming (MIP) models [3,13,17,20,23,32]. While a combinatorial branch-and-bound method appears to be the currently fastest and most robust method for small and moderately sized graphs in practice, we aim at revealing the limitations and structural weaknesses of present MIP formulations and at improving their competitiveness. In this respect, the contribution of this paper is threefold.

First, we derive two MIP formulations that represent the current state-of-the-art. Then, we reveal one essential reason why these formulations are of poor utility in solving larger or more difficult instances to optimality. More precisely, we show that, for *any* given graph, the lower bound on its pathwidth obtained when solving their linear programming (LP) relaxations, i.e., the linear program that arises when neglecting any integrality requirements on the variables, is *zero*. Second, we propose a new MIP formulation that uses different variables than the previous ones, is compact in size, and also exhibits a slightly different perspective on the problem. Compared to the conference proceedings version [28] of this paper, we describe this new formulation more easily accessible, formally verify its correctness, do not only show experimentally but also give structural proofs that its LP relaxation yields stronger lower bounds than the two other representatives, and present further valid inequalities that can be used as cutting planes. Finally, we evaluate the performance of the now three representatives when passed to a MIP solver in order to solve the problem to optimality and discuss the obtained results based on the structure of the used benchmark instances.

The outline of this article is as follows. Sect. 2 introduces the vertex separation problem formally. Sect. 3 summarizes related research about the vertex separation and pathwidth problems with emphasis on existing MIP and other practical solution methods. It provides the basis to derive representative MIP models reflecting the current state-of-the-art in Sect. 4. In Sect. 5, we present our novel MIP formulation as well as extensions and structural results connected to it. The experimental evaluations are described in Sect. 6 and the paper closes with a conclusion in Sect. 7.

2. The vertex separation problem

Let $G = (V, A)$ be a directed graph (digraph) and let $\Pi(V)$ be the set of all permutations of the vertices V of G. For a given permutation or *linear order* $\pi \in \Pi(V)$, we denote with $\pi(v)$ the position of each $v \in V$ in π and consider the sets $L(\pi, v) = \{u \in V \mid \pi(u) \leq \pi(v)\}$ and $R(\pi, v) = \{w \in V \mid \pi(v) < \pi(w)\}$. They can be thought of as being generated by a cut $\delta(\pi, v)$ through the linear order defined by π that is carried out marginally close to the right of v. This is illustrated in Fig. 1 for an example graph. With each cut $\delta(\pi, v)$, we associate its corresponding *separation* $S(\pi, v) = \{w \in R(\pi, v) \mid \exists u \in L(\pi, v) : (u, w) \in A\}$, i.e., informally, the subset of vertices in $R(\pi, v)$ that are 'hit' by arcs coming in from vertices in $L(\pi, v)$. For a fixed linear order $\pi \in \Pi(V)$, let $vs(\pi, G) = \max_{v \in V} |S(\pi, v)|$ be the corresponding maximum vertex separation. The *vertex separation problem* is then to find a linear order $\pi^* \in \Pi(V)$, such that the maximum vertex separation is minimum, i.e., $vs(\pi^*, G) \leq vs(\pi, G)$ for all $\pi \in \Pi(V)$. The value $vs(G) = vs(\pi^*, G)$ is also referred to as the *vertex separation number* of G. For an undirected graph $G = (V, E)$, the vertex separation associated to a linear order $\pi \in \Pi(V)$ is $vs(\pi, G) = \max_{v \in V} |\{w \in R(\pi, v) \mid \exists u \in L(\pi, v) : \{u, w\} \in E\}|$. Clearly, this value can be computed using a digraph-based method by replacing each edge $\{u, v\} \in E$ by two arcs $(u, v) \in A$ and $(v, u) \in A$.

As a remark, the previous definitions differ from other presentations in that L, R, S and the cuts are associated with a *vertex* rather than with a *position* in the linear order. While this makes no difference concerning the objective, it does in modeling, as we will see in the following sections.

3. Related work

The amount of literature dealing with tree- and pathwidth as a theoretic concept is tremendous, so in the following, we restrict ourselves to research that deals with the exact solution of the pathwidth or vertex separation problem. As

before, we consider undirected graphs $G = (V, E)$, and digraphs $G = (V, A)$ and, throughout the paper, we have $n = |V|$ and, respectively, $m = |E|$ or $m = |A|$.

The proposal of combinatorial branch-and-bound algorithms started with the work of Solano and Pióro [32] in the context of wavelength-division multiplexing in optical communication networks. Coudert et al. later improved and extended their method by preprocessing and pruning techniques [14]. Comprehensive experimental studies in these articles show that it is currently the fastest and most robust method to tackle the pathwidth problem in practice. Many instances with up to about 100 vertices can be solved routinely (still depending on the graph structure), and also some considerably larger ones. There is an implementation of their algorithm in SageMath[1] and also one of the dynamic programming algorithm presented in [12] (that is applicable to graphs with up to 31 vertices only). A further combinatorial branch-and-bound scheme was proposed by Fraire Huacuja et al. [20]. On the more theoretical side, enumerative algorithms whose running times can be bounded by $\mathcal{O}(1.89^n)$ and $\mathcal{O}(1.9657^n)$ have respectively been given by Kitsunai et al. [25], and Suchan and Villanger [33], and exponential-time dynamic programming or fixed-parameter algorithms are presented in [7,12,19,22].

Among the existing MIP formulations, we distinguish two types of binary variables that have been used to model linear orders, namely *position assignment* and *set assignment variables*. Their formal definition will be given in Sect. 4. We are particularly interested in the *strength* of a formulation. A common strength measure is the lower bound on the objective (which is here the optimum vertex separation) obtained when solving a formulation's LP relaxation, i.e., the linear program that arises when neglecting any integrality constraints on the variables.

The first MIP formulation has been proposed by Solano and Pióro [32]. It is set assignment-based, has $3n^2 + 1 = \mathcal{O}(n^2)$ variables and $\mathcal{O}(nm)$ constraints. Computational experience with this model is hardly reported. However, a slightly adapted version has been implemented in SageMath and used for experiments, e.g., in [14,21]. As already noticed by Coudert in [13], its number of variables can be reduced to $2n^2 + 1$. Moreover, this reduction comes without losing strength, and it is easily seen that only n^2 of the variables need their integrality to be enforced explicitly as integrality of the others is then implied. The resulting formulation will be considered for experiments and stated formally as model MIP_S in Sect. 4. As proposed in [21], the number of constraints could be reduced to $\mathcal{O}(n^2)$ as well but the corresponding changes require to explicitly enforce integrality of all but one variable in the corresponding MIP while being counterproductive in terms of strength.

Duarte et al. [17] devised the first MIP formulation with position assignment variables (and a variable neighborhood search heuristic). It has $\mathcal{O}(n^4)$ variables and constraints. The quartic size stems from a straightforward linearization of quadratic terms. Using a more compact linearization and by creating some variables arc- instead of vertex-pair-based, the number of variables and constraints can be reduced to $\mathcal{O}(n^2 m)$ and $\mathcal{O}(nm)$, respectively [20]. Another position assignment model by Gurski [23] involves $\mathcal{O}(n^6)$ variables which is impractical. However, as discussed by Coudert [13], it is possible to formulate a position assignment model with only $\mathcal{O}(n^2)$ variables and $\mathcal{O}(nm)$ constraints. Compared to the one by Duarte et al., its relaxation strength is at least as good whence we will consider it in our comparisons and state it as model MIP_P in the next section.

Finally, another integer program for undirected graphs has been presented by Biedl et al. [3]. Its main motivation was to show that the pathwidth problem can be formulated within a more general framework that assigns vertices and edges to grid coordinates. Due to a poor performance in their experiments, it was transformed into a satisfiability problem where graphs with $n + m < 45$ could almost always be solved, but those where $n + m > 70$ only rarely.

4. Representative MIP models

As announced in Sect. 3, we now formulate two models, MIP_P and MIP_S, as the most compact representatives for and with the same LP relaxation strength as the existing ones with either position or set assignment variables. We then show that their relaxations (and thus those of any model represented by them) have a *zero objective* for *any* given digraph,[2] i.e., provide worst-possible lower bounds for any digraph that has a non-zero pathwidth.

4.1. Position assignments (MIP_P)

We start with the representative model MIP_P for position assignments. It models a linear order π in the same way as the models in [17,20,23] but introduces, as proposed by Coudert [13], fewer variables. These are the following:

- $a_{v,p} = 1$ if $\pi(v) = p$ and $a_{v,p} = 0$ otherwise (position assignment variables).
- $c_{v,p} = 1$ if $\pi(v) > p$ and there is a vertex u with $\pi(u) \leq p$ such that $(u, v) \in A$ (v 'counts' for the cut at position p), and $c_{v,p} = 0$ otherwise.
- Z: The objective variable that captures the vertex separation number.

[1] SageMath is an open-source mathematics library. http://www.sagemath.org.

[2] The model in [20] can be reformulated such that its relaxation yields non-zero bounds for some graphs, but it remains of too excessive size and inferior to MIP_P in practice.

The full model MIP_P is then:

$$\begin{aligned}
&\min Z \\
&\text{s.t. } \textstyle\sum_{p=1}^{n} a_{v,p} && = 1 && \text{for all } v \in V && (1)\\
&\textstyle\sum_{v\in V} a_{v,p} && = 1 && \text{for all } 1 \le p \le n && (2)\\
&\textstyle\sum_{q=1}^{p} (a_{u,q} - a_{v,q}) && \le c_{v,p} && \text{for all } v \in V,\ (u,v) \in A,\ 1 \le p \le n && (3)\\
&\textstyle\sum_{v\in V} c_{v,p} && \le Z && \text{for all } 1 \le p \le n && (4)\\
&a_{v,p} && \in \{0,1\} && \text{for all } v \in V,\ 1 \le p \le n && (5)\\
&c_{v,p} && \in [0,1] && \text{for all } v \in V,\ 1 \le p \le n && (6)\\
&Z && \ge 0 && && (7)
\end{aligned}$$

Equations (1) and (2) enforce a bijective mapping of vertices to positions. Inequalities (3) require $c_{v,p}$ to be equal to one whenever $\pi(v) > p$ and there is a vertex u with $\pi(u) \le p$ and $(u,v) \in A$. Finally, the objective value Z is given by the maximum sum $\sum_{v\in V} c_{v,p}$ over all positions p (inequalities (4)). The number of variables is $2n^2 + 1$ and thus of order $\mathcal{O}(n^2)$, and the number of constraints is $3n + nm$ and thus of order $\mathcal{O}(nm)$.

4.2. Set assignments (MIP_S)

The representative model MIP_S for the set-assignment approach arises from the initial one by Solano and Pióro [32] when successively applying the improvements made in the SageMath-implementation and those proposed by Coudert in [13]. It constructs a linear order π of the vertices V by enforcing a collection of sets $\mathcal{W} = \{W_1, \ldots, W_n\}$ such that $W_p \subseteq V$ and $|W_p| = p$ for each $p \in \{1, \ldots, n\}$, and $W_p \subset W_q$ for any $p < q$. That is, W_1 specifies the vertex $v \in V$ with $\pi(v) = 1$, and in general, the rank $\pi(v)$ of vertex $v \in V$ is $\pi(v) = \min\{p \mid v \in W_p\}$.

Model MIP_S adopts the c- and objective variables from MIP_P but replaces the a-variables with variables $b_{v,p} = 1$ if $v \in W_p$ and $b_{v,p} = 0$ otherwise. Then the lines (1)–(3) and (5) of MIP_P are replaced by the following ones:

$$\begin{aligned}
&b_{v,p} - b_{v,p+1} && \le 0 && \text{for all } v \in V,\ 1 \le p \le n-1 && (8)\\
&\textstyle\sum_{v\in V} b_{v,p} && = p && \text{for all } 1 \le p \le n && (9)\\
&b_{u,p} - b_{v,p} && \le c_{v,p} && \text{for all } v \in V,\ (u,v) \in A,\ 1 \le p \le n && (10)\\
&b_{v,p} && \in \{0,1\} && \text{for all } v \in V,\ 1 \le p \le n && (11)
\end{aligned}$$

Inequalities (8) are *forwarding constraints* implementing the condition that $v \in W_q$ for any $q > p$ if $v \in W_p$. Equations (9) require the sets W_p to have their desired cardinalities. The inequalities (10) force $c_{v,p}$ to be equal to one whenever $v \notin W_p$ and there is a vertex $u \in W_p$ with $(u,v) \in A$. The number of variables is the same as in model MIP_P and thus again of order $\mathcal{O}(n^2)$. The number of constraints is $n(n-1) + n + nm + n$, i.e., a little larger but still of order $\mathcal{O}(nm)$.

4.3. Relaxation strength of MIP_P and MIP_S

We denote the LP relaxations of MIP_P and MIP_S with LP_P and LP_S. Further, we write $MIP_P(G)$ ($MIP_S(G)$) or $LP_P(G)$ ($LP_S(G)$) when referring to the respective concrete MIP or LP associated with a given digraph $G = (V, A)$.

Theorem 4.1. *For any digraph $G = (V, A)$, there exists an optimum solution to $LP_P(G)$ and $LP_S(G)$ with objective value zero.*

Proof. We explicitly construct respective feasible solutions $(a^*, c^*, Z^*)^T$ and $(b^*, c^*, Z^*)^T$ to $LP_P(G)$ and $LP_S(G)$ with $a^*, b^*, c^* \in [0,1]^{n^2}$ and $Z^* = 0$, for any digraph $G = (V, A)$ with $|V| = n$.

In case of $LP_P(G)$, set $a^*_{v,p} = \frac{1}{n}$ for each $v \in V$ and $1 \le p \le n$. It is easily verified that equations (1) and (2) are satisfied. For any arc $(u,v) \in A$ and inequality of type (3), we obtain a left hand side of zero since $a^*_{u,q}$ equals $a^*_{v,q}$. Hence, each of these constraints imposes $c^*_{v,p} \ge 0$ which leads to $c^*_{v,p} = 0$ and thus $Z^* = 0$ as we are minimizing.

Considering $LP_S(G)$, set $b^*_{v,p} = \frac{p}{n}$ for each $v \in V$ and $1 \le p \le n$. This satisfies equations (9) and inequalities (8) strictly. Since inequalities (10) comprise only variables on their left hand sides that refer to the same position p, they again all evaluate to $c^*_{v,p} \ge 0$ such that an optimum solution has $Z^* = 0$. □

5. A novel linear-ordering model (MIP_L)

While the vertex separation problem has often been explained in a linear ordering context, we are not aware of any promising mixed-integer programming formulation that is based on what is commonly known as *linear ordering variables* in the literature [29]. The only exception is [13] where such a formulation is indicated but then overcomplicated leading to unsatisfactory results.

Linear ordering variables $x_{v,w} \in \{0,1\}$ are defined for each pair of vertices $v, w \in V$, $v \neq w$, with the interpretation that, in any feasible solution, $\pi(v) < \pi(w)$ if and only if $x_{v,w} = 1$ [29]. This will make it particularly easy to exploit the following properties within a corresponding new model MIP_L.

Lemma 5.1. *Let $v, w \in V$, $v \neq w$, and $(v, w) \in A$. Then $w \in S(\pi, v)$ if and only if $\pi(v) < \pi(w)$.*

Proof. Since v is the rightmost vertex of $L(\pi, v)$, w must be in $R(\pi, v)$ if it succeeds v. As we assume that $(v, w) \in A$, $w \in S(\pi, v)$ then follows immediately. Conversely, if $w \in S(\pi, v)$, then w must succeed v in π by definition. □

Lemma 5.2. *Let $v, w \in V$, $v \neq w$, and $(v, w) \notin A$. Then $w \in S(\pi, v)$ if and only if $\pi(v) < \pi(w)$ and there is some $u \neq v, w$ such that $\pi(u) < \pi(v)$ and $(u, w) \in A$.*

Proof. First, suppose $w \in S(\pi, v)$. Then $w \in R(\pi, v)$ and hence $\pi(w) > \pi(v)$. Assume now that there is no such u as required. Then the only vertex in $L(\pi, v)$ that could cause $w \in S(\pi, v)$ is v itself. However, by assumption, $(v, w) \notin A$ which yields a contradiction. The converse direction is again immediate from the definition of $S(\pi, v)$. □

Another central difference of MIP_L compared to MIP_P and MIP_S is that it defines cuts w.r.t. a *vertex* rather than a *position*. This leads to a replacement of the previous c-variables by new vertex-relational y-variables where $y_{v,w} = 1$ if $w \in S(\pi, v)$. We now formulate MIP_L entirely:

$$\min Z$$

$$\text{s.t.} \quad x_{v,w} + x_{w,v} = 1 \qquad \text{for all } v, w \in V, v < w \tag{12}$$

$$x_{u,v} + x_{v,w} + x_{w,u} \leq 2 \qquad \text{for all } u, v, w \in V, u \neq v \neq w \neq u \tag{13}$$

$$x_{u,v} + x_{v,w} - y_{v,w} \leq 1 \qquad \text{for all } u, v, w \in V, v \neq w \neq u, (v, w) \notin A, (u, w) \in A \tag{14}$$

$$\sum_{\substack{v \neq w, \\ (v,w) \notin A}} y_{v,w} + \sum_{\substack{v \neq w, \\ (v,w) \in A}} x_{v,w} \leq Z \qquad \text{for all } v \in V \tag{15}$$

$$Z \geq 0$$

$$x_{v,w} \in \{0,1\} \qquad \text{for all } v, w \in V, v \neq w$$

$$y_{v,w} \in [0,1] \qquad \text{for all } v, w \in V, v \neq w, (v, w) \notin A$$

Stated like this, MIP_L has respectively $n_x = 2\binom{n}{2}$ and $n_y = 2\binom{n}{2} - m$ linear ordering (x-) and vertex separation (y-) variables which is of order $\mathcal{O}(n^2)$. However, variables $x_{w,v}$ where $w > v$ are defined only for convenience in expressing constraints. In practice any of their occurrences can be replaced by $1 - x_{v,w}$ and the equations (12) then be eliminated. Further, variables $y_{v,w}$ may be omitted (or fixed to zero) for vertices $w \in V$ that have no incoming arcs. Due to the *three-dicycle inequalities* (13), there are $\mathcal{O}(n^3)$ constraints in total. Moreover, the following additional valid inequalities w.r.t. variables $y_{v,w}$ may be formulated straightforwardly:

$$y_{v,w} - x_{v,w} \leq 0 \qquad \text{for all } v, w \in V, v \neq w, (v, w) \notin A \tag{16}$$

$$y_{v,w} - \sum_{\substack{u \neq v, \\ (u,w) \in A}} x_{u,v} \leq 0 \qquad \text{for all } v, w \in V, v \neq w, (v, w) \notin A \tag{17}$$

The first one basically states that $y_{v,w}$ must be equal to zero whenever $x_{v,w}$ is, and the second one states that $y_{v,w}$ must be zero whenever all the variables $x_{u,v}$ such that $(u, w) \in A$ are. Similar constraints could be formulated for MIP_P and MIP_S as well. Like for these, they are however not necessary in order to have a valid characterization of integer feasible solutions that correspond to feasible solutions of the vertex separation problem as we shall prove now.

Theorem 5.1. *For any digraph $G = (V, A)$, any feasible solution to $MIP_L(G)$ corresponds to a feasible solution to the vertex separation problem associated with G and vice versa. Moreover, the variable Z reflects an upper bound on the maximum vertex separation imposed by this solution, and attains the exact vertex separation number $vs(G)$ if it is an* optimal *solution.*

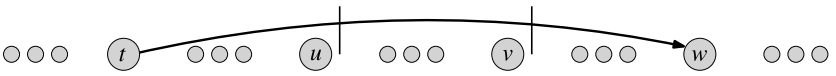


Fig. 2. Illustration of the 'implication' that a cut implication inequality forwards from $y_{u,w}$ to $y_{v,w}$ if v is placed between u and w.

Proof. Let $(x^*, y^*, Z^*)^T$ with $x^* \in \{0,1\}^{n_x}$, $y^* \in [0,1]^{n_y}$, and $Z^* \in \mathbb{R}^{\geq 0}$, be a given feasible solution to $MIP_L(G)$. Since x^* is integer, and satisfies the three-dicycle-inequalities (13), it uniquely defines a linear vertex ordering $\pi^* \in \Pi(V)$ [29] which actually *is* a (unique) feasible solution to the vertex separation problem associated with G. We now consider the objective function: Let $v, w \in V$, $v \neq w$. Suppose first that $(v,w) \in A$. Then, by Lemma 5.1, we have $w \in S(\pi^*, v)$ if and only if $x^*_{v,w} = 1$. Now suppose that $(v,w) \notin A$. Then, since inequalities (14) implement Lemma 5.2, $y^*_{v,w} = 1$ holds whenever there is a vertex u such that $(u,w) \in A$, $\pi^*(u) < \pi^*(v)$, $\pi^*(w) > \pi^*(v)$, and thus $w \in S(\pi^*, v)$. Otherwise, we have $0 \leq y^*_{v,w} \leq 1$. Thus, by inequalities (15), it follows that $Z^* \geq \sum_{\substack{v \neq w,\\ (v,w) \notin A}} y^*_{v,w} + \sum_{\substack{v \neq w,\\ (v,w) \in A}} x^*_{v,w} \geq |S(\pi^*, v)|$, for each $v \in V$. In particular, if $(x^*, y^*, Z^*)^T$ is optimum, there is a vertex $v \in V$ for which both inequalities hold strictly, since Z is minimized and there are no restrictions apart from inequalities (14) enforcing any $y^*_{v,w}$ to a value greater than zero. As a remark, strictness could also be enforced for *any* feasible solution by adding inequalities (16) and (17).

For the reverse direction, suppose we are given a feasible solution to the vertex separation problem associated with G and with underlying linear vertex order π^*. For each pair $v, w \in V$, $v \neq w$, set $x^*_{v,w} = 1$ if $\pi^*(v) < \pi^*(w)$ and $x^*_{v,w} = 0$ otherwise. Since π^* is acyclic by definition, x^* satisfies inequalities (13). Now let $v, w \in V$, $v \neq w$ and $(v,w) \notin A$. If there is a vertex u such that $(u,w) \in A$, $\pi^*(u) < \pi^*(v)$, and $\pi^*(w) > \pi^*(v)$, set $y^*_{v,w} = 1$. Otherwise, set $y^*_{v,w} = 0$. This satisfies all inequalities (14). Finally, set $Z^* = \sum_{\substack{v \neq w,\\ (v,w) \notin A}} y^*_{v,w} + \sum_{\substack{v \neq w,\\ (v,w) \in A}} x^*_{v,w} = vs(\pi^*, G)$ to obtain a corresponding feasible solution $(x^*, y^*, Z^*)^T$ to $MIP_L(G)$. □

5.1. Cutting planes for MIP_L

The cubic number of three-dicycle inequalities slows down solving the linear programming relaxation LP_L of MIP_L for midsize or large instances in practice. They are a natural candidate to be considered as cutting planes [29], i.e., to be omitted in the initial LP and then, as long as there exist inequalities that are violated by an LP solution, these are iteratively added and the resulting LP is resolved. When neglecting them at first, inequalities (14) become dominant in terms of size. However, their number can be bounded by $\mathcal{O}(n(n^2 - m))$, i.e., their number decreases with increasing arc density.

Further candidates to be considered as cutting planes are all inequalities that are satisfied by any integer solution $(x^*, y^*, Z^*)^T$ to LP_L, but may be violated by a fractional one. As a remark, this is true for any further inequality being valid for the linear ordering problem (plenty exist, see e.g. [29]). We will now draw attention to a problem-specific class of valid inequalities for MIP_L which we call *cut implication inequalities*.

They can be considered a truncated version of inequalities (14) for the case where not only $(v,w) \notin A$ but $(u,w) \notin A$ as well. Their underlying idea is to 'transitively forward' information whether $w \in S(\pi^*, u)$ (captured by $y_{u,w}$) to conclude on whether $w \in S(\pi^*, v)$ as well (captured by $y_{v,w}$) based on whether v is placed between u and w (see Fig. 2 for an illustration). Cut implication inequalities are thus of the following form:

$$y_{u,w} + x_{v,w} - x_{v,u} - y_{v,w} \leq 1 \qquad \text{for all } u, v, w \in V, u \neq v \neq w \neq u, \; (u,w) \notin A, (v,w) \notin A \tag{18}$$

Theorem 5.2. *Let $G = (V, A)$ be a digraph and $(x^*, y^*, Z^*)^T$ a feasible solution to $MIP_L(G)$ that in addition satisfies inequalities (17). Then it satisfies as well all cut implication inequalities (18).*

Proof. Assume to the contrary that $(x^*, y^*, Z^*)^T$ violates an implication cut inequality for some given $u, v, w \in V$ as specified. It then immediately follows that $x^*_{v,w} = 1$, $x^*_{v,u} = 0$, and $y^*_{u,w} - y^*_{v,w} > 0$. We thus have $\pi^*(u) < \pi^*(v) < \pi^*(w)$, and hence also $x^*_{u,w} = 1$. Since inequalities (17) hold and $y^*_{u,w} > 0$, there must be some vertex $t \in V$, $t \notin \{u, v, w\}$, such that $(t,w) \in A$, $\pi^*(t) < \pi^*(u)$, and thus $y^*_{u,w} \geq \underbrace{x^*_{u,w} + x^*_{t,u} - 1}_{=1}$. However, at the same time, we have $y^*_{v,w} \geq \underbrace{x^*_{v,w} + x^*_{t,v} - 1}_{=1}$ and this contradicts $y^*_{u,w} - y^*_{v,w} > 0$. □

As a remark, it is trivial to devise a feasible solution to MIP_L not satisfying inequalities (17) which then also violates a cut implication inequality – since then there is no upper bound restriction apart from $y_{v,w} \leq 1$ on each such variable. Also, non-existence of the arc (u,w) is specified as a requirement, since an implication cut inequality would be dominated by an inequality $x_{u,v} + x_{v,w} - y_{v,w} \leq 1$ of type (14) if $y_{u,w}$ is replaced by $x_{u,w}$. This is shown by the following calculation:

$$\begin{aligned} & x_{u,w} + x_{v,w} - x_{v,u} - y_{v,w} \leq 1 \\ \Leftrightarrow \quad & x_{u,w} + x_{v,w} - (1 - x_{u,v}) - y_{v,w} \leq 1 \\ \Leftrightarrow \quad & x_{u,w} + x_{v,w} + x_{u,v} - y_{v,w} \leq 2 \\ \Leftrightarrow \quad & x_{u,v} + x_{v,w} - y_{v,w} \leq \underbrace{2 - x_{u,w}}_{\geq 1} \end{aligned}$$

What remains to show is that there actually *are* optimum fractional solutions to LP_L which are cut off by implication cut inequalities.

Theorem 5.3. *There exist digraphs $G = (V, A)$ and optimal solutions to $LP_L(G)$ (satisfying as well inequalities (16) and (17)), that violate an implication cut inequality, even for $|V| = 4$ and $|A| = 2$.*

Proof. We prove this claim by explicitly displaying such an optimal solution for $LP_L(G)$ associated to the graph $G = (V, A)$ with $V = \{1, 2, 3, 4\}$ and $A = \{\{1, 2\}, \{2, 1\}\}$. Because of the two-dicycle, an optimum solution to $LP_L(G)$ has objective value $\frac{1}{2}$ which results if and only if $x_{1,2} = \frac{1}{2}$. A corresponding optimum (basic) solution to $LP_L(G)$ is $x_{1,2} = x_{2,3} = \frac{1}{2}$, $x_{1,3} = 1$, $x_{1,4} = x_{2,4} = x_{3,4} = 0$, $y_{3,2} = y_{4,3} = \frac{1}{2}$, and $y_{1,3} = y_{1,4} = y_{2,3} = y_{2,4} = y_{3,1} = y_{3,4} = y_{4,1} = y_{4,2} = 0$. It violates the implication cut inequality $y_{4,3} - y_{1,3} + x_{1,3} - x_{1,4} \leq 1$ by $\frac{1}{2}$. □

5.2. Relaxation strength of LP_L

While we could show that the optimum objective of LP_P and LP_S is zero for any given input digraph $G = (V, A)$, we will now show that this is the case for LP_L if and only if the vertex separation number of G is zero. Moreover, we will show that if G is a complete digraph on n vertices, then the objective value of $LP_L(G)$ is $\frac{n-1}{2}$ (while the vertex separation number is $n - 1$).

Observation 5.1. *The vertex separation number $vs(G)$ of a digraph $G = (V, A)$ is equal to zero if and only if G is acyclic.*

Theorem 5.4. *The objective value of $LP_L(G)$ is zero if and only if the digraph $G = (V, A)$ is acyclic.*

Proof. Let $G = (V, A)$ be an acyclic digraph. Then, by Observation 5.1 and Theorem 5.1, an optimum solution to $MIP_L(G)$ has objective zero. Since the optimum objective value of $LP_L(G)$ is a lower bound on the one of $MIP_L(G)$, it must be zero, too.

Now let G have an arbitrary simple cycle C with $2 \leq k \leq |V|$ vertices. We may assume w.l.o.g. that the vertices are indexed by $1, \ldots, k$, i.e., the arcs of C are $(1, 2), (2, 3), \ldots, (k - 1, k), (k, 1)$. Now since, by inequalities (15), $Z \geq x_{i,j}$ for each $(i, j) \in C$, $Z = 0$ implies $x_{i,j} = 0$ for all $(i, j) \in C$. However, by equations (12), it then follows immediately that $x_{j,i} = 1$ for all $(i, j) \in C$. If $k = 3$, then the three-dicycle-inequality $\underbrace{x_{3,2} + x_{2,1} + x_{1,3}}_{=3} \leq 2$ is clearly violated. For $k = 4$, one of $\underbrace{x_{2,1} + x_{1,4}}_{=2} + x_{4,2} \leq 2$ and $\underbrace{x_{4,3} + x_{3,2}}_{=2} + x_{2,4} \leq 2$ must be. And for $k \geq 5$, either $\underbrace{x_{2,1} + x_{1,k}}_{=2} + x_{k,2} \leq 2$ is violated, or the argument can be inductively applied to the cyclic order $k, \ldots, 2, k$ with $k - 1$ distinct vertices. We conclude that a solution as displayed must violate at least one of the three-dicycle-inequalities (13). Hence, in a solution of $LP_L(G)$, $x_{i,j} > 0$ must hold for at least one arc contained in C and thus $Z > 0$ as well. □

Theorem 5.5. *If $G = (V, A)$ is the complete digraph on $|V| = n$ vertices then the objective value of $LP_L(G)$ is $\frac{n-1}{2}$.*

Proof. Since G is complete, $LP_L(G)$ contains no y-variables, no inequalities of type (14), and, for each $v \in V$, inequalities (15) read $Z \geq \sum_{v \neq w} x_{v,w}$. Moreover, due to equations (12), we have $\sum_{v \in V} \left(\sum_{v \neq w} x_{v,w} \right) = \sum_{v \in V} \left(\sum_{v < w} (x_{v,w} + x_{w,v}) \right) = |V| \frac{|V|-1}{2}$. Now let x^* be an optimal solution to $LP_L(G)$ and suppose there is a vertex $v \in V$ such that $\sum_{v \neq w} x^*_{v,w} < \frac{|V|-1}{2}$. Then, by a simple counting argument, there must be a different vertex $u \neq v$ such that $\sum_{u \neq w} x^*_{u,w} > \frac{|V|-1}{2}$. This proves that $Z \geq \frac{|V|-1}{2}$. Clearly, an objective $Z = \frac{|V|-1}{2}$ results by setting $x_{v,w} = \frac{1}{2}$ for all $v, w \in V$, $v \neq w$, and this solution trivially satisfies all constraints of $LP_L(G)$ – whence it is a feasible and optimal one. □

Apart from these results, the gap between the optimum objective of $LP_L(G)$ and $MIP_L(G)$ can be made arbitrarily bad (e.g., if G is an n-by-n-grid graph, which has pathwidth n but where LP_L provides a constant lower bound of 2, as the experiments in Sect. 6 show). This is not surprising (even though, in general, a solution to LP_L cannot be easily rounded to

Table 1
Results for the TWLIB instance test bed.

Instance	n	m	pw	LB	Time [s]			Time [s] or LB		
				LP_L	LP_L	LP_P	LP_S	MIP_L	MIP_P	MIP_S
barley	48	252	7	3.5850	1.66	4.39	0.80	4.0000*	2.0000*	4.3226*
david	87	812	13	7.2287	34.92	171.87	41.98	8.0000*	0.0000*	1.0000*
huck	74	602	10	5.6045	13.58	61.56	8.59	6.0000*	0.0000*	1.0000*
mainuk	48	396	7	4.7705	0.90	6.84	1.27	6.0000*	1.0000*	5.0000*
mildew	35	160	5	2.6246	0.18	0.74	0.20	4.0000*	1.0000*	176.65
myciel2	5	5	2	1.0000	0.00	0.00	0.00	0.00	0.01	0.00
myciel3	11	20	5	2.0833	0.00	0.00	0.00	1.75	1.10	0.34
myciel4	23	71	10	3.8580	0.06	0.15	0.04	5.2817*	7.2000*	147.44
myciel5	47	236	20	6.8750	2.18	8.02	1.20	8.0000*	1.0000*	6.0000*
queen5_5	25	320	18	6.6370	0.09	0.42	0.09	7.9205*	392.46	13.5714*
queen6_6	36	580	25	8.5000	0.85	3.65	0.54	8.9608*	1.0000*	8.0000*
queen7_7	49	952	35	10.4246	8.50	22.54	2.03	10.5145*	1.0000*	7.0000*
queen8_8	64	1456	45	12.2308	36.99	106.08	4.65	13.0000*	0.0000*	6.0000*
queen8_12	96	2736	65	15.5303	1790.72	1042.17	61.18	–	–	2.0000*
queen9_9	81	2112	58	14.1075	272.16	391.01	16.93	15.0000*	0.0000*	1.0000*
queen10_10	100	2940	72	16.0686	1348.58	1335.73	68.30	–	–	2.0000*
water	32	246	10	4.9286	0.36	0.93	0.21	5.7253*	1.0000*	9.0000*

feasible one with the same objective) in light of the fact that, unless $\mathcal{P} = \mathcal{NP}$, there is no polynomial-time approximation algorithm for the pathwidth problem with an absolute error of at most $|V|^{\epsilon}$ for any $0 < \epsilon < 1$, as has been shown by Bodlaender et al. [9].

6. Experimental evaluation

In the previous sections, we derived three different MIP models, MIP_P, MIP_S and MIP_L together with their LP relaxations LP_P, LP_S, and LP_L. Our experiments shall give insights about the following questions:

(1) How good are the lower bounds on the pathwidth obtained with LP_L on differently structured graphs?
(2) How do LP_P, LP_S, and LP_L compare in terms of solution times?
(3) How do MIP_P, MIP_S and MIP_L perform with a sophisticated solver?

We compile a testbed of bidirected 147 graphs with at most 100 vertices from benchmark sets previously used for comparisons. First, we consider a set TWLIB consisting of 17 graphs from the TreewidthLIB. Second, we select six GRID and 20 TREE instances from the VSPLIB [17]. Then, there are 20 graphs whose bidirected arcs represent the non-zero off-diagonal entries of instances from the Harwell–Boeing sparse matrix collection (denoted HB) [18], and another set of 84 instances with only 16–24 vertices called SMALL that were introduced in [30].

All LPs and MIPs were solved using version 12.6 of CPLEX.[3] For maximum fairness and to reflect the effects of each formulation as purely as possible, all its internal cutting plane algorithms, heuristics, and presolve routines were disabled. Each run was executed single-threadedly on a Debian Linux machine with an Intel Core i7-3770T processor running at 2.5 GHz and with 32 GB RAM.

6.1. Results

There is a table for each considered benchmark subset. Table 1 gives the results for the set TWLIB, Table 2 for the set HB, Table 3 for the TREE and GRID graphs, and finally Tables 4 and 5 for the SMALL instances. In case of the trees, we restricted the displayed results to four instances with indices 1, 6, and 11 as these are representative for all the graphs with indices 1–5, 6–10 and 11–15, respectively. Each of the tables reports the following information: The instance name, the number of vertices n, the number of directed arcs m and the pathwidth of the graph. Then, in column five, the lower bound obtained with LP_L is given. Columns 6–8 display the time required for solving the respective LP relaxations. The time specified for LP_L is the total time for iteratively solving LPs and adding three-dicycle-inequalities until a solution is found where none of these are violated. The last three columns give the results for solving MIP_L, MIP_P, and MIP_S. If an instance could be solved within the time limit of ten minutes (wall clock time) by a particular formulation then the time needed is given in the respective table cell. Otherwise, the lower bound on the pathwidth at termination is displayed and marked with an asterisk. We now discuss the three posed research questions one-by-one.

[3] IBM ILOG CPLEX Optimization Studio is a proprietary LP and MIP solver.

Table 2
Results for the HB instance test bed.

Instance	n	m	pw	LB	Time [s]			Time [s] or LB		
				LP_L	LP_L	LP_P	LP_S	MIP_L	MIP_P	MIP_S
ash85	85	438	8	3.4521	42.93	74.31	13.75	4.0000*	1.0000*	1.0000*
bcspwr01	39	92	4	1.5000	0.19	0.74	0.29	3.0000*	1.0000*	426.94
bcspwr02	49	118	3	1.7769	0.85	2.39	0.63	133.53	1.0000*	234.58
bcsstk01	48	352	13	4.7545	3.34	6.12	1.02	5.0000*	1.0000*	5.0000*
bcsstk02	66	4290	65	32.5000	5.85	389.99	28.52	32.5000*	0.0000*	2.0000*
can___24	24	136	5	3.0625	0.04	0.21	0.05	83.70	47.70	9.29
can___61	61	496	8	4.2394	3.51	22.77	2.52	5.0000*	1.0000*	3.0000*
can___62	62	156	4	1.8333	1.96	7.55	1.96	3.0000*	1.0000*	2.0000*
can___73	73	304	10	3.5556	17.60	27.88	4.01	4.0000*	1.0000*	4.0000*
can___96	96	672	13	4.0000	58.16	203.11	37.10	5.0000*	0.0000*	2.0000*
curtis54	54	248	6	2.9552	4.51	6.54	1.05	4.0000*	1.0000*	3.0000*
dwt___59	59	208	5	2.1667	2.23	7.82	1.48	3.0000*	1.0000*	4.0000*
dwt___66	66	254	2	1.9999	6.65	16.11	3.19	93.04	1.0000*	320.26
dwt___72	72	150	3	1.4473	4.44	13.66	3.24	115.68	1.0000*	208.56
dwt___87	87	454	8	3.8142	79.59	89.42	10.25	4.0000*	1.0000*	2.0000*
ibm32	32	180	10	3.9028	0.27	0.66	0.15	5.0000*	2.5227*	8.0000*
pores_1	30	206	7	4.1000	0.27	0.54	0.12	5.0000*	5.2381*	63.71
nos4	100	494	8	3.0000	51.18	161.99	37.92	4.0000*	1.0000*	4.0000*
steam3	80	848	7	5.4860	41.94	133.86	14.03	6.0000*	1.0000*	1.0000*
will57	57	254	5	2.8749	1.47	8.68	1.66	4.0000*	1.0000*	3.0000*

Table 3
Results for the TREE and GRID instance test bed.

Instance	n	m	pw	LB	Time [s]			Time [s] or LB		
				LP_L	LP_L	LP_P	LP_S	MIP_L	MIP_P	MIP_S
tree_22_rot1	22	42	3	1.1250	0.01	0.04	0.04	32.14	1.9167*	12.81
tree_67_rot1	67	132	4	1.2500	1.32	8.14	2.49	2.0000*	1.0000*	2.0000*
tree_67_rot6	67	132	4	1.2656	1.09	7.80	2.43	2.0000*	1.0000*	2.0000*
tree_67_rot11	67	132	4	1.2812	1.51	7.68	2.49	2.0000*	1.0000*	2.0000*
grid_5	25	80	5	2.0000	0.04	0.16	0.04	78.16	552.65	12.60
grid_6	36	120	6	2.0000	0.22	0.72	0.22	4.0000*	2.3077*	57.28
grid_7	49	168	7	2.0000	0.61	3.17	0.61	2.2222*	1.0000*	277.32
grid_8	64	224	8	2.0000	1.21	13.13	2.19	2.0000*	1.0000*	4.7913*
grid_9	81	288	9	2.0000	4.08	42.08	9.29	3.0000*	1.0000*	2.0779*
grid_10	100	360	10	2.0000	20.08	125.20	23.88	2.0000*	1.0000*	2.0000*

6.1.1. How good are the lower bounds on the pathwidth obtained with LP_L?

The quality of the lower bounds obtained with LP_L largely depends on the graph structure. GRID graphs embody a particularly problematic class. For the 2-dimensional ones considered in the experiment, LP_L delivers a constant lower bound of 2 although an n-by-n grid graph has pathwidth n. Thus, by further increasing their size, the lower bound quality can be made arbitrarily weak. Also, for the TREE instances that are very similar to each other, the lower bounds obtained are relatively weak and differ only marginally when the true pathwidth increases. As was proven structurally, the lower bound quality is 50% if G is the complete digraph, like in case of `bcsstk02` from the HB instances. Apart from this exception, the other instances from the HB set reflect sparse graphs (between 2 and 25% of the possible arcs exist). Often, they have a (possibly slightly incomplete) canonical bidirected Hamiltonian path, and a few shorter 'shortcutting' subpaths that represent a banded matrix structure. Across these, and also the SMALL instances, the lower bound quality is about 50% on average, but can be considerably worse (e.g., 30% for `can____96`) or much better (even 99% for `dwt____66`). In case of the TWLIB instances, the picture is similar and, apart from the `queen`-graphs, they also have a similar density between 8 and 25%. For the `queen`-graphs with a density between about 30% and 54%, the quality of the lower bound drops considerably – with an increasing number of vertices and decreasing density. However, as one can also see from the results, in general, very low or high densities and the (ir)regularity of vertex degree patterns can lead to both, better or worse, results.

6.1.2. How do LP_P, LP_S, and LP_L compare in terms of solution times?

Apart from a few exceptions, one can say that, across the instances considered, LP_S can be solved the fastest, followed by LP_L, while solving LP_P usually takes considerably more time. There are some outliers where solving LP_L takes much longer, especially for `queen_8_12` and `queen_10_10` where even the time limit destined for the MIP experiment is exceeded. These extremes diminish when activating the presolve methods of CPLEX. Iterative addition of the three-dicycle inequalities as cutting planes is essential in order to achieve moderate solution times for LP_L. If all of them are added in advance, the solution times often degrade unsatisfactorily.

Table 4
Results for the SMALL instance test bed (part one).

Instance	n	m	pw	LB	Time [s]			Time [s] or LB		
				LP_L	LP_L	LP_P	LP_S	MIP_L	MIP_P	MIP_S
p17_16_24	16	48	4	2.0833	0.01	0.04	0.02	1.79	13.66	0.64
p18_16_21	16	42	3	1.7273	0.01	0.02	0.01	0.53	4.73	0.41
p19_16_19	16	38	3	1.5081	0.01	0.02	0.01	0.46	22.29	0.54
p20_16_18	16	36	3	1.3333	0.00	0.02	0.01	1.00	68.30	0.87
p21_17_20	17	40	3	1.6000	0.01	0.02	0.01	0.42	28.12	0.41
p22_17_19	17	38	2	1.4265	0.01	0.02	0.01	0.24	2.80	0.41
p23_17_23	17	46	3	1.6400	0.01	0.02	0.01	0.36	6.50	0.46
p24_17_29	17	58	4	2.0208	0.01	0.02	0.01	5.08	32.66	0.85
p25_17_20	17	40	3	1.5000	0.01	0.02	0.01	0.33	21.79	0.81
p26_17_19	17	38	2	1.4000	0.01	0.02	0.01	0.23	4.02	0.29
p27_17_19	17	38	3	1.4242	0.01	0.02	0.01	0.43	65.64	0.89
p28_17_18	17	36	2	1.2396	0.00	0.02	0.01	0.20	3.64	0.22
p29_17_18	17	36	2	1.2364	0.00	0.02	0.01	0.14	5.42	0.36
p30_17_19	17	38	3	1.5000	0.01	0.02	0.01	0.57	167.88	0.87
p31_18_21	18	42	2	1.4667	0.01	0.03	0.01	0.29	3.43	0.44
p32_18_20	18	40	3	1.4000	0.00	0.02	0.01	0.40	38.02	1.10
p33_18_21	18	42	3	1.5000	0.01	0.02	0.02	0.50	40.58	0.63
p34_18_21	18	42	2	1.5000	0.00	0.02	0.01	0.26	1.86	0.58
p35_18_19	18	38	2	1.2946	0.01	0.03	0.01	0.40	4.00	0.48
p36_18_20	18	40	3	1.4000	0.01	0.02	0.01	0.64	261.47	0.70
p37_18_20	18	40	3	1.4000	0.01	0.02	0.01	0.50	69.64	1.39
p38_18_19	18	38	2	1.2027	0.01	0.02	0.02	0.31	2.48	0.56
p39_18_19	18	38	2	1.2625	0.00	0.02	0.02	0.27	2.16	0.36
p40_18_32	18	64	5	2.3333	0.02	0.04	0.02	26.07	42.48	2.25
p41_19_20	19	40	2	1.2321	0.01	0.03	0.02	0.22	5.33	0.41
p42_19_24	19	48	4	1.7000	0.01	0.04	0.02	3.26	381.26	2.53
p43_19_22	19	44	3	1.5000	0.01	0.03	0.02	0.98	14.93	1.08
p44_19_25	19	50	4	1.8000	0.02	0.04	0.02	5.55	47.74	2.24
p45_19_25	19	50	3	1.7171	0.01	0.05	0.02	0.34	10.61	1.78
p46_19_20	19	40	2	1.3000	0.02	0.03	0.02	0.48	4.81	0.54
p47_19_21	19	42	3	1.5000	0.02	0.03	0.02	0.65	334.28	1.54
p48_19_21	19	42	2	1.3333	0.00	0.03	0.02	0.30	7.35	0.61
p49_19_22	19	44	3	1.5000	0.01	0.03	0.02	0.70	14.31	1.14
p50_19_25	19	50	3	1.6429	0.01	0.04	0.02	0.60	28.03	0.79
p51_20_28	20	56	4	1.9500	0.02	0.05	0.02	8.08	33.08	1.76
p52_20_27	20	54	3	1.7500	0.02	0.04	0.02	1.10	19.04	1.61
p53_20_22	20	44	2	1.3562	0.01	0.04	0.02	0.24	3.72	0.45
p54_20_28	20	56	3	1.8214	0.01	0.05	0.02	0.66	45.14	1.82
p55_20_24	20	48	3	1.5833	0.02	0.04	0.02	0.89	86.20	1.18
p56_20_23	20	46	4	1.5000	0.01	0.04	0.02	5.84	3.0000*	7.89
p57_20_24	20	48	3	1.6667	0.01	0.04	0.02	0.69	22.47	1.20
p58_20_21	20	42	3	1.3111	0.01	0.04	0.02	1.00	196.32	3.52
p59_20_23	20	46	3	1.4667	0.01	0.04	0.02	0.98	90.87	2.09
p60_20_22	20	44	3	1.3636	0.01	0.04	0.02	0.87	359.81	1.26
p61_21_22	21	44	3	1.3308	0.02	0.04	0.02	0.89	2.0000*	3.86
p62_21_30	21	60	4	2.0417	0.02	0.06	0.03	7.97	107.18	2.19
p63_21_42	21	84	6	2.6939	0.03	0.07	0.02	207.49	5.0000*	10.48
p64_21_22	21	44	2	1.3083	0.02	0.05	0.02	0.36	8.90	0.56
p65_21_24	21	48	3	1.5000	0.01	0.05	0.02	0.76	118.15	1.42
p66_21_28	21	56	3	1.7750	0.02	0.05	0.02	0.49	119.24	0.94
p67_21_22	21	44	2	1.3069	0.01	0.04	0.03	0.95	19.48	1.34
p68_21_27	21	54	3	1.7143	0.02	0.05	0.03	1.21	38.06	0.52

6.1.3. How do MIP_P, MIP_S, and MIP_L perform with a sophisticated solver?

Most (73 of 84) SMALL instances could be solved easily with all formulations. Also in case of the other instances, optimum or near-optimal solutions were typically found quickly with all formulations, but often optimality cannot be proven within the time limit. Concerning the number of solved instances and derived lower bounds after ten minutes of computation time, MIP_P is inferior to at least one of the other two models except for seven instances in total. MIP_S solves a few instances more than MIP_L within the time limit. On the other hand, for many unsolved instances, MIP_L provides a better lower bound than MIP_S when the timeout occurs. However, further investigations revealed that the branch-and-bound scheme of CPLEX is able to improve the global lower bound much better when solving MIP_S than when solving MIP_L. More precisely, MIP_L starts with a non-zero bound but the bound then often stagnates early and solving the LPs takes more time compared to MIP_S which starts with a zero bound that can then often be improved steadily and the occurring subproblems can be enumerated quicker. So when raising the time limit, MIP_S solves or obtains the best bound for even slightly more instances although still many remain unsolved even after 60 minutes.

Table 5
Results for the SMALL instance test bed (part two).

Instance	n	m	pw	LB	Time [s]			Time [s] or LB		
				LP_L	LP_L	LP_P	LP_S	MIP_L	MIP_P	MIP_S
p69_21_23	21	46	3	1.4000	0.01	0.05	0.03	0.94	2.0000*	6.66
p70_21_25	21	50	3	1.7000	0.02	0.05	0.02	0.84	24.78	1.18
p71_22_29	22	58	4	1.8500	0.02	0.07	0.03	12.4	455.29	3.47
p72_22_49	22	98	7	3.0625	0.03	0.13	0.04	5.2433*	234.648	3.04
p73_22_29	22	58	4	1.7500	0.02	0.07	0.02	17.54	3.0000*	13.27
p74_22_30	22	60	3	1.8600	0.02	0.07	0.04	0.72	30.52	2.28
p75_22_25	22	50	3	1.6000	0.02	0.06	0.03	3.61	147.64	1.68
p76_22_30	22	60	3	1.8056	0.02	0.07	0.03	0.55	25.91	2.24
p77_22_37	22	74	5	2.3333	0.04	0.08	0.03	73.88	282.31	1.77
p78_22_31	22	62	4	1.9028	0.02	0.07	0.02	10.58	3.0000*	2.04
p79_22_29	22	58	3	1.8056	0.02	0.07	0.03	0.76	165.22	1.16
p80_22_30	22	60	4	1.8644	0.02	0.06	0.03	8.66	2.3541*	2.53
p81_23_46	23	92	7	2.6429	0.04	0.11	0.04	4.7158*	5.8377*	14.27
p82_23_24	23	48	2	1.3137	0.02	0.07	0.03	8.19	10.84	3.08
p83_23_24	23	48	2	1.3326	0.02	0.07	0.04	1.36	7.10	2.22
p84_23_26	23	52	3	1.4917	0.03	0.07	0.04	1.82	272.78	1.52
p85_23_26	23	52	3	1.3854	0.02	0.08	0.04	3.53	1.6667*	15.32
p86_23_24	23	48	2	1.2458	0.02	0.08	0.04	0.32	11.34	1.74
p87_23_30	23	60	4	1.8723	0.03	0.08	0.04	10.02	317.16	1.21
p88_23_26	23	52	3	1.5000	0.03	0.07	0.04	1.92	103.46	2.52
p89_23_27	23	54	3	1.5000	0.02	0.08	0.03	10.32	402.06	0.94
p90_23_35	23	70	4	2.1389	0.03	0.09	0.03	12.43	81.03	21.96
p91_24_33	24	66	4	1.9077	0.04	0.11	0.05	28.20	410.66	21.12
p92_24_26	24	52	3	1.5000	0.02	0.09	0.04	1.54	126.46	1.99
p93_24_27	24	54	2	1.5116	0.03	0.09	0.04	1.10	16.87	1.28
p94_24_31	24	62	4	1.9375	0.03	0.10	0.05	10.47	486.80	3.38
p95_24_27	24	54	3	1.6000	0.04	0.09	0.04	0.87	385.19	3.70
p96_24_27	24	54	3	1.5090	0.03	0.08	0.04	2.33	319.06	3.53
p97_24_26	24	52	2	1.3707	0.02	0.09	0.04	0.81	16.08	0.88
p98_24_29	24	58	3	1.7576	0.03	0.14	0.05	2.08	176.97	4.82
p99_24_27	24	54	3	1.6000	0.03	0.10	0.04	1.41	142.83	4.46
p100_24_34	24	68	4	2.0000	0.03	0.10	0.04	10.30	3.0000*	40.28

7. Conclusion

In this paper, we shed light on the fact that the linear programming relaxations of existing mixed-integer programs for the vertex separation problem provide worst possible lower bounds on the pathwidth of a directed graph. This is one of the major reasons why these are of poor utility in computing the pathwidth of certain graph classes in practice.

We then proposed a new mixed-integer program MIP_L for the vertex separation problem based on linear ordering variables as a first step towards stronger linear programming relaxations that provide useful lower bounds for a branch-and-bound scheme. Superiority in this respect has been proven formally and evaluated experimentally, comparing it to two other models MIP_P and MIP_S representing the present state-of-the-art formulations that employ either position or set assignment variables.

It became evident that the LP relaxation of MIP_L provides better lower bounds than those of MIP_S and MIP_P, and could still be solved relatively quickly in most of the cases. This advantage does however not necessarily translate into a better MIP solution performance. Contrarily, MIP_S could typically be solved the fastest (as also its relaxation) and could prove optimality for the highest number of instances within the time limit of ten minutes. MIP_P appeared to be inferior to both other models.

However, based on our experiments using a commercial MIP solver, none of them is yet competitive to the currently best combinatorial branch-and-bound methods to which many of the unsolved instances pose no challenge. Moreover, even the lower bounds obtained with MIP_L are often inferior to those achieved by combinatorial methods for treewidth (see, e.g., [11]). To become competitive, a prospective MIP formulation needs to better exploit structural knowledge to combine a good lower bound quality with effective search space reductions while retaining a fast relaxation performance.

References

[1] S. Arnborg, A. Proskurowski, Linear time algorithms for NP-hard problems restricted to partial k-trees, Discrete Appl. Math. 23 (1) (1989) 11–24, https://doi.org/10.1016/0166-218X(89)90031-0.

[2] S. Arnborg, D.G. Corneil, A. Proskurowski, Complexity of finding embeddings in a k-tree, SIAM J. Algebraic Discrete Methods 8 (2) (1987) 277–284, https://doi.org/10.1137/0608024.

[3] T.C. Biedl, T. Bläsius, B. Niedermann, M. Nöllenburg, R. Prutkin, I. Rutter, Using ILP/SAT to determine pathwidth, visibility representations, and other grid-based graph drawings, CoRR abs/308.6778v2.

[4] H.L. Bodlaender, Dynamic programming on graphs with bounded treewidth, in: T. Lepistö, A. Salomaa (Eds.), Proc. 15th Intern. Colloq. on Automata, Languages and Programming, in: LNCS, vol. 317, Springer, 1988, pp. 105–118.
[5] H.L. Bodlaender, A tourist guide through treewidth, Acta Cybern. 11 (1–2) (1993) 1–23.
[6] H.L. Bodlaender, A partial k-arboretum of graphs with bounded treewidth, Theor. Comput. Sci. 209 (1–2) (1998) 1–45, https://doi.org/10.1016/S0304-3975(97)00228-4.
[7] H.L. Bodlaender, T. Kloks, Efficient and constructive algorithms for the pathwidth and treewidth of graphs, J. Algorithms 21 (2) (1996) 358–402, https://doi.org/10.1006/jagm.1996.0049.
[8] H.L. Bodlaender, A.M.C.A. Koster, Combinatorial optimization on graphs of bounded treewidth, Comput. J. 51 (3) (2007) 255, https://doi.org/10.1093/comjnl/bxm037.
[9] H. Bodlaender, J. Gilbert, H. Hafsteinsson, T. Kloks, Approximating treewidth, pathwidth, frontsize, and shortest elimination tree, J. Algorithms 18 (2) (1995) 238–255, https://doi.org/10.1006/jagm.1995.1009.
[10] H. Bodlaender, J. Gustedt, J.A. Telle, Linear-time register allocation for a fixed number of registers, in: Proc. 9th Annual ACM-SIAM Symp. on Discrete Algorithms, SODA '98, SIAM, Philadelphia, PA, USA, 1998, pp. 574–583.
[11] H.L. Bodlaender, T. Wolle, A.M.C.A. Koster, Contraction and treewidth lower bounds, J. Graph Algorithms Appl. 10 (1) (2006) 5–49, https://doi.org/10.1007/978-3-540-30140-0_56.
[12] H.L. Bodlaender, F.V. Fomin, A.M. Koster, D. Kratsch, D.M. Thilikos, A note on exact algorithms for vertex ordering problems on graphs, Theory Comput. Syst. 50 (3) (2012) 420–432, https://doi.org/10.1007/s00224-011-9312-0.
[13] D. Coudert, A Note on Integer Linear Programming Formulations for Linear Ordering Problems on Graphs, Tech. Rep. hal-01271838, INRIA, Feb 2016.
[14] D. Coudert, D. Mazauric, N. Nisse, Experimental evaluation of a branch-and-bound algorithm for computing pathwidth and directed pathwidth, J. Exp. Algorithmics 21 (2016) 1.3:1–1.3:23, https://doi.org/10.1145/2851494.
[15] N. Deo, M.S. Krishnamoorthy, M.A. Langston, Exact and approximate solutions for the gate matrix layout problem, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 6 (1) (1987) 79–84, https://doi.org/10.1109/TCAD.1987.1270248.
[16] J. Díaz, J. Petit, M. Serna, A survey of graph layout problems, ACM Comput. Surv. 34 (3) (2002) 313–356, https://doi.org/10.1145/568522.568523.
[17] A. Duarte, L.F. Escudero, R. Martí, N. Mladenovic, J.J. Pantrigo, J. Sánchez-Oro, Variable neighborhood search for the vertex separation problem, Comput. Oper. Res. 39 (12) (2012) 3247–3255, https://doi.org/10.1016/j.cor.2012.04.017.
[18] I.S. Duff, R.G. Grimes, J.G. Lewis, Sparse matrix test problems, ACM Trans. Math. Softw. 15 (1) (1989) 1–14, https://doi.org/10.1145/62038.62043.
[19] J.A. Ellis, I.H. Sudborough, J.S. Turner, The vertex separation and search number of a graph, Inf. Comput. 113 (1) (1994) 50–79, https://doi.org/10.1006/inco.1994.1064.
[20] H.J. Fraire Huacuja, N. Castillo-García, R.A. Pazos Rangel, J.A. Martínez Flores, J.J. González Barbosa, J.M. Carpio Valadez, Two new exact methods for the vertex separation problem, Int. J. Comb. Optim. Probl. Inform. 6 (1) (2015) 31–41.
[21] H.J. Fraire-Huacuja, N. Castillo-García, M.C. López-Locés, J.A. Martínez Flores, R.A. Pazos R., J.J. González Barbosa, J.M. Carpio Valadez, Integer linear programming formulation and exact algorithm for computing pathwidth, in: P. Melin, O. Castillo, J. Kacprzyk (Eds.), Nature-Inspired Design of Hybrid Intelligent Systems, Springer, 2017, pp. 673–686.
[22] M. Fürer, Faster computation of path-width, in: V. Mäkinen, S.J. Puglisi, L. Salmela (Eds.), Proc. 27th Intern. Workshop on Combinatorial Algorithms, Springer, Cham, 2016, pp. 385–396.
[23] F. Gurski, Linear programming formulations for computing graph layout parameters, Comput. J. 58 (11) (2015) 2921–2927, https://doi.org/10.1093/comjnl/bxv013.
[24] N.G. Kinnersley, The vertex separation number of a graph equals its path-width, Inf. Process. Lett. 42 (6) (1992) 345–350, https://doi.org/10.1016/0020-0190(92)90234-M.
[25] K. Kitsunai, Y. Kobayashi, K. Komuro, H. Tamaki, T. Tano, Computing directed pathwidth in $O(1.89^n)$ time, in: D.M. Thilikos, G.J. Woeginger (Eds.), Proc. 7th Intern. Symp. on Param. and Exact Comput, in: LNCS, vol. 7535, Springer, 2012, pp. 182–193.
[26] Y. Kobayashi, K. Komuro, H. Tamaki, Search space reduction through commitments in pathwidth computation: An experimental study, Proc. 13th Intern. Symp. on Experimental Algorithms, vol. 8504, Springer, 2014, pp. 388–399.
[27] T. Lengauer, Black–white pebbles and graph separation, Acta Inform. 16 (4) (1981) 465–475, https://doi.org/10.1007/BF00264496.
[28] S. Mallach, Linear ordering based MIP formulations for the vertex separation or pathwidth problem, in: L. Brankovic, J. Ryan, W.F. Smyth (Eds.), Combinatorial Algorithms, Springer, Cham, 2018, pp. 327–340.
[29] R. Martí, G. Reinelt, The Linear Ordering Problem, Springer, 2011.
[30] R. Martí, V. Campos, E. Piñana, A branch and bound algorithm for the matrix bandwidth minimization, Eur. J. Oper. Res. 186 (2) (2008) 513–528, https://doi.org/10.1016/j.ejor.2007.02.004.
[31] N. Robertson, P.D. Seymour, Graph minors. II. Algorithmic aspects of tree-width, J. Algorithms 7 (3) (1986) 309–322, https://doi.org/10.1016/0196-6774(86)90023-4.
[32] F. Solano, M. Pióro, Lightpath reconfiguration in WDM networks, J. Opt. Commun. Netw. 2 (12) (2010) 1010–1021, https://doi.org/10.1364/JOCN.2.001010.
[33] K. Suchan, Y. Villanger, Computing pathwidth faster than 2^n, in: J. Chen, F.V. Fomin (Eds.), Proc. 4th Intern. Symp. on Param. and Exact Comput, in: LNCS, vol. 5917, Springer, 2009, pp. 324–335.
[34] B. Yang, Y. Cao, Digraph searching, directed vertex separation and directed pathwidth, Discrete Appl. Math. 156 (10) (2008) 1822–1837, https://doi.org/10.1016/j.dam.2007.08.045.